

Streaming API Responses

A Complete Guide for AI Engineers — Basic to Advanced

Learn how to stream real-time token-by-token responses from Anthropic's Claude API using Python — from your first chunk to production-ready async endpoints.

Reference: docs.anthropic.com/en/api/streaming

Server-Sent Events SSE protocol deep dive	8 Code Examples Basic → async → FastAPI	Extended Thinking Streaming Claude's reasoning
---	---	--

Table of Contents

1.	What Is Streaming and Why Use It?	3
2.	How Streaming Works — The SSE Protocol	3
3.	Your First Streaming Call	4
4.	Anatomy of a Streaming Event	4
5.	Accumulating Content and Checking Stop Reason	5
6.	The Stream Context Manager (.stream())	5
7.	Streaming Tool / Function Calls	6
8.	Async Streaming	7
9.	Building a FastAPI Streaming Endpoint	8
10	Error Handling and Retries	9
.		
11	Tracking Token Usage During Streaming	9
.		
12	Extended Thinking (Streaming Claude's Reasoning)	10
.		
13	Best Practices Cheat Sheet	10
.		

1. What Is Streaming and Why Use It?

By default, the Anthropic API waits until Claude finishes generating every token, then sends the complete response in one HTTP response. For short answers this is fine, but for long outputs — code generation, essays, detailed analysis — the user stares at a blank screen for several seconds before anything appears.

Streaming flips this model: tokens are sent to your application the moment Claude produces them, just like a person typing in real time. Your app can render each word as it arrives, making the interaction feel **instant**.

Non-streaming vs. Streaming

Non-Streaming	Streaming
Waits for full response	Tokens arrive in real time
Single HTTP response	Chunked transfer / SSE
Higher perceived latency	Lower perceived latency
Simple to parse	Requires event iteration
Good for short outputs	Essential for long outputs

2. How Streaming Works — The SSE Protocol

Anthropic uses **Server-Sent Events (SSE)**, a standard web protocol where the server keeps an HTTP connection open and pushes data to the client. Each SSE event has a named type and a JSON data payload:

Raw SSE wire format from Anthropic API

```
event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"Hello"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" world"}}

event: message_stop
data: {"type":"message_stop"}
```

Unlike OpenAI's single-field chunks, Anthropic uses a **structured event model** with distinct event types for each phase of the response. The Python SDK handles all SSE parsing for you — you just iterate events or use the convenient `text_stream` helper.

■ **Note:** You never need to parse SSE manually when using the official anthropic Python library. The SDK handles chunked decoding, JSON parsing, and event sequencing automatically.

3. Setup and Your First Streaming Call

Install the library and initialise the client:

setup.py

```
# Install the Anthropic Python library
# pip install anthropic

import anthropic

client = anthropic.Anthropic(api_key="YOUR_ANTHROPIC_API_KEY")
# Or set the ANTHROPIC_API_KEY environment variable and call anthropic.Anthropic()
```

Now compare a non-streaming call with a streaming one:

no_streaming.py — full response arrives at once

```
# Non-streaming: waits for the FULL response before returning
response = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Write a short poem about Python."}]
)

# The entire text arrives at once
print(response.content[0].text)
```

basic_streaming.py — tokens arrive one by one

```
# Streaming: tokens arrive one chunk at a time
with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Write a short poem about Python."}],
) as stream:
    for text in stream.text_stream:
        # text_stream yields plain string content directly
        print(text, end="", flush=True)

print() # newline after streaming finishes
```

The key change: use `client.messages.stream()` as a context manager instead of `client.messages.create()`. The `text_stream` property yields plain strings, filtering out all non-text events automatically.

4. Anatomy of a Streaming Event

Anthropic's streaming model is **event-based**, not chunk-based. A full stream follows a predictable sequence of typed events, each corresponding to a phase of the response lifecycle.

```
event_anatomy.py — inspect the raw event stream

# Use stream=True on messages.create() to see raw SSE events
with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=256,
    messages=[{"role": "user", "content": "Say hello."}],
) as stream:
    for event in stream:
        print(event)

# The event stream follows this order:
#
# RawMessageStartEvent    — message shell with empty content, input token count
# RawContentBlockStartEvent(index=0, content_block=TextBlock(text='', type='text'))
# RawContentBlockDeltaEvent(index=0, delta=TextDelta(text='Hello', type='text_delta'))
# RawContentBlockDeltaEvent(index=0, delta=TextDelta(text='!', type='text_delta'))
# ...more deltas...
# RawContentBlockStopEvent(index=0)
# RawMessageDeltaEvent — stop_reason='end_turn', cumulative output token count
# RawMessageStopEvent   — stream is done
```

Event Type	When It Fires	Key Fields
message_start	Once, at the very beginning	Message shell, input token count
content_block_start	Start of each content block	index, content_block type (text / tool_use / thinking)
content_block_delta	Each new token or chunk	delta.type: text_delta input_json_delta thinking_delta
content_block_stop	End of each content block	index of the block that finished
message_delta	Once, near the end	stop_reason, cumulative output token count
message_stop	Once, at the very end	Signals the stream is complete

ping	Occasionally, during long streams	Keep-alive — no data, safe to ignore
------	-----------------------------------	--------------------------------------

5. Accumulating Content and Checking Stop Reason

In many applications you need both: print tokens as they arrive AND have the complete text when the stream ends. The pattern below collects text chunks into a string and checks the stop reason afterwards.

accumulate.py

```
# Pattern: stream tokens to the screen AND collect the full text
collected_text = ""
chunk_count     = 0

with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "List 3 benefits of streaming APIs."}],
) as stream:
    for text in stream.text_stream:
        collected_text += text
        chunk_count    += 1
        print(text, end="", flush=True)

print()
print(f"\nTotal characters : {len(collected_text)}")
print(f"Total text chunks: {chunk_count}")
```

stop_reason.py — always check why the stream ended

```
# Check stop_reason from the final message to know WHY the stream ended
with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Explain recursion briefly."}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

# After the 'with' block the stream is closed; get the final typed message:
final = stream.get_final_message()
print(f"\n\nStop reason      : {final.stop_reason}")
# Possible stop_reason values:
#   'end_turn'      → model finished naturally
#   'max_tokens'    → hit the max_tokens limit
#   'stop_sequence' → a custom stop sequence was matched
#   'tool_use'      → model wants to call a tool
```


6. The Stream Context Manager (.stream())

The SDK's `.stream()` context manager is the recommended way to consume a streaming response. It provides three key conveniences:

- **text_stream** — yields plain string content, skipping all non-text events automatically.
- **get_final_message()** — returns a fully-typed `Message` object (identical to `.create()`) after the loop.
- **Automatic cleanup** — the HTTP connection is closed cleanly even if you break early.

stream_manager.py — preferred pattern for most use cases

```
# .stream() context manager – the preferred pattern
# Provides text_stream, get_final_message(), and safe cleanup

with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Explain async I/O in Python."}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

# After the with-block, retrieve the fully-typed Message object:
final = stream.get_final_message()
print(f"\n\nStop reason    : {final.stop_reason}")
print(f"Input tokens   : {final.usage.input_tokens}")
print(f"Output tokens  : {final.usage.output_tokens}")
print(f"Model           : {final.model}")
```

■ **Note:** Use `.stream()` context manager for new code. Use `messages.create(stream=True)` only when you need to iterate raw SSE events manually (e.g. to handle `thinking_delta` or `input_json_delta`).

7. Streaming Tool / Function Calls

When Claude decides to call a tool, the function name and JSON arguments stream as `input_json_delta` events containing `partial_json` strings. You must buffer them and parse the complete JSON only after the content block closes.

tool_streaming.py — assemble tool arguments from delta events

```

import json

tools = [{
    "name": "get_weather",
    "description": "Get the current weather for a city",
    "input_schema": {
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "City and state, e.g. San Francisco, CA"
            }
        },
        "required": ["location"]
    }
}]

args_buffer = ""
function_name = None

with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "What is the weather in Tokyo?"}],
) as stream:
    for event in stream:
        # Tool input arrives as 'input_json_delta' events
        if hasattr(event, "type") and event.type == "content_block_delta":
            delta = event.delta
            if delta.type == "input_json_delta":
                args_buffer += delta.partial_json
                print(delta.partial_json, end="", flush=True)

        # content_block_start tells us the tool name
        if hasattr(event, "type") and event.type == "content_block_start":
            cb = event.content_block
            if cb.type == "tool_use":
                function_name = cb.name
                print(f"\nClaude is calling tool: {function_name}")

    final = stream.get_final_message()
    if final.stop_reason == "tool_use":
        parsed_args = json.loads(args_buffer)
        print(f"\nTool      : {function_name}")
        print(f"Arguments : {json.dumps(parsed_args, indent=2)}")

```

■■ **Warning:** Always JSON-parse the `args_buffer` after the content block ends (`stop_reason='tool_use'`). Each `partial_json` fragment is incomplete JSON — parsing mid-stream will raise an error.

8. Async Streaming

For web servers, chatbots, and any I/O-bound application use AsyncAnthropic combined with `async for`. This keeps your event loop unblocked while waiting for tokens from the API.

async_streaming.py — non-blocking streaming with asyncio

```
import asyncio
import anthropic

async_client = anthropic.AsyncAnthropic(api_key="YOUR_API_KEY")

async def stream_response(prompt: str) -> str:
    """Stream a response and return the full text."""
    full_text = ""

    async with async_client.messages.stream(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        async for text in stream.text_stream:
            print(text, end="", flush=True)
            full_text += text

    print()
    return full_text

async def stream_multiple(prompts: list[str]):
    """Stream multiple completions CONCURRENTLY."""
    tasks = [stream_response(p) for p in prompts]
    results = await asyncio.gather(*tasks)
    return results

# Run it
asyncio.run(stream_response("What is the event loop in Python?"))
```

Using `asyncio.gather()` lets you stream from multiple prompts truly concurrently — all requests run in parallel on the same event loop, with no threads required.

9. Building a FastAPI Streaming Endpoint

The most common production pattern is a backend that proxies Claude tokens directly to a browser or mobile client via SSE. FastAPI's `StreamingResponse` makes this straightforward with `AsyncAnthropic`.

main.py — FastAPI SSE streaming endpoint

```

# pip install fastapi uvicorn anthropic
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import anthropic
import asyncio

app = FastAPI()
client = anthropic.AsyncAnthropic(api_key="YOUR_API_KEY")

async def token_generator(prompt: str):
    """Yield SSE-formatted chunks for the browser."""
    async with client.messages.stream(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        async for text in stream.text_stream:
            # Server-Sent Event format: 'data: <payload>\n\n'
            yield f"data: {text}\n\n"

        yield "data: [DONE]\n\n"

@app.get("/stream")
async def stream_endpoint(prompt: str):
    return StreamingResponse(
        token_generator(prompt),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no", # Disable nginx buffering
        },
    )

# Run: uvicorn main:app --reload
# Test: curl "http://localhost:8000/stream?prompt=Hello+world"

```

The browser consumes this endpoint with the native EventSource API or the `@microsoft/fetch-event-source` library for POST support. The `X-Accel-Buffering: no` header prevents nginx from buffering the entire response before forwarding it to the client.

10. Error Handling and Retries

Streaming connections can fail mid-response. Always wrap streaming calls in error handling. Anthropic's API returns an `overloaded_error` (HTTP 529) during peak load — this should be retried with backoff.

error_handling.py — retry with exponential backoff

```
import anthropic
import time

def stream_with_retry(prompt: str, max_retries: int = 3) -> str:
    """Stream with exponential backoff on transient errors."""
    for attempt in range(max_retries):
        try:
            full_text = ""
            with client.messages.stream(
                model="claude-opus-4-5",
                max_tokens=1024,
                messages=[{"role": "user", "content": prompt}],
            ) as stream:
                for text in stream.text_stream:
                    print(text, end="", flush=True)
                    full_text += text
            print()
            return full_text

        except anthropic.APIConnectionError as e:
            print(f"\nConnection error (attempt {attempt+1}): {e}")
        except anthropic.RateLimitError as e:
            wait = 2 ** attempt
            print(f"\nRate limited – retrying in {wait}s...")
            time.sleep(wait)
        except anthropic.APIStatusError as e:
            print(f"\nAPI error {e.status_code}: {e.message}")
            if e.status_code >= 500:
                time.sleep(2 ** attempt)  # retry on 5xx (overloaded)
            else:
                break  # don't retry on 4xx

    return ""

result = stream_with_retry("Explain Python generators in two sentences.")
```

■ **Note:** For Claude 4.6+ models, a mid-stream recovery is also possible: save the partial response, then send a new user message saying 'Continue from where you left off.'

11. Tracking Token Usage During Streaming

Unlike OpenAI's API, **Anthropic always includes token usage** in the `message_delta` event — no extra flag is needed. Access it via `get_final_message()` after the stream ends. The counts are cumulative and reflect the complete request.

`token_usage.py` — usage stats are always available

```
# Token usage is included in the message_delta event (cumulative)
# Access it via get_final_message() after the stream ends

with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "What is machine learning?"}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

# Usage is always available — no extra flag needed (unlike OpenAI)
final = stream.get_final_message()
print()
print(f"\nInput tokens      : {final.usage.input_tokens}")
print(f"Output tokens       : {final.usage.output_tokens}")
print(f"Total tokens         : {final.usage.input_tokens + final.usage.output_tokens}")
```

12. Extended Thinking — Streaming Claude's Reasoning

Anthropic's Claude models support **extended thinking**: the model reasons through a problem step-by-step before giving its final answer. When streaming is enabled, this reasoning arrives as `thinking_delta` events — a unique capability not available in other APIs.

`extended_thinking.py` — stream Claude's reasoning and final answer

```

# Extended thinking – Claude reasons before answering
# Thinking tokens stream as 'thinking_delta' events

with client.messages.stream(
    model="claude-opus-4-5",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{
        "role": "user",
        "content": "What is the greatest common divisor of 1071 and 462?"
    }],
) as stream:
    for event in stream:
        if event.type == "content_block_delta":
            delta = event.delta

            if delta.type == "thinking_delta":
                # Claude's internal reasoning – stream in grey/italic in your UI
                print(f"[thinking] {delta.thinking}", end="", flush=True)

            elif delta.type == "text_delta":
                # Claude's final answer
                print(delta.text, end="", flush=True)

final = stream.get_final_message()
print(f"\n\nStop reason    : {final.stop_reason}")
print(f"Input tokens   : {final.usage.input_tokens}")
print(f"Output tokens  : {final.usage.output_tokens}")

```

■ **Note:** Thinking tokens count toward your max_tokens budget. Set budget_tokens high enough for complex problems (e.g. 10,000+). Thinking blocks cannot be partially recovered during error recovery — only text blocks can be resumed.

13. Best Practices Cheat Sheet

Situation	Recommendation
Simple scripts / notebooks	Use <code>.stream()</code> context manager with <code>text_stream</code> .
Production apps / APIs	Use <code>.stream()</code> with <code>AsyncAnthropic</code> for non-blocking I/O.
Web endpoints (REST/SSE)	Use <code>FastAPI StreamingResponse</code> + <code>text/event-stream</code> content type.
Need full message after streaming	Call <code>stream.get_final_message()</code> after the <code>with</code> block.
Streaming tool calls	Buffer <code>delta.partial_json</code> ; parse JSON on <code>stop_reason='tool_use'</code> .
Token counting	Always available — read <code>final.usage.input_tokens</code> / <code>output_tokens</code> .
Error resilience	Catch <code>APIConnectionError</code> + <code>RateLimitError</code> ; retry 529 (overloaded) with backoff.
Nginx / reverse proxy	Set <code>X-Accel-Buffering: no</code> header to prevent response buffering.
Streaming Claude's reasoning	Enable extended thinking and handle <code>thinking_delta</code> events separately.
Concurrent streams	Use <code>asyncio.gather()</code> with <code>AsyncAnthropic</code> for parallel requests.
Mid-stream error recovery	Save partial text; for Claude 4.6+ ask model to continue in a new user message.